esprit Se former autrement	Année Universitaire : 2025-2026 **Applications Web Distribuées** **UP_WEB**

Atelier-Création de Microservice

Gestion des candidats

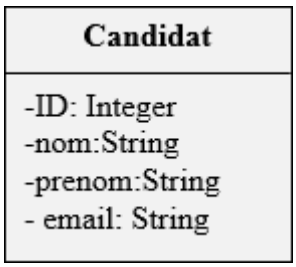
Objectifs

Notre objectif est de créer un Microservice qui fournit des exemples d'opérations.

Développement des micro-services

Nous visons à créer un premier microservice de gestion des candidats dans une entreprise.

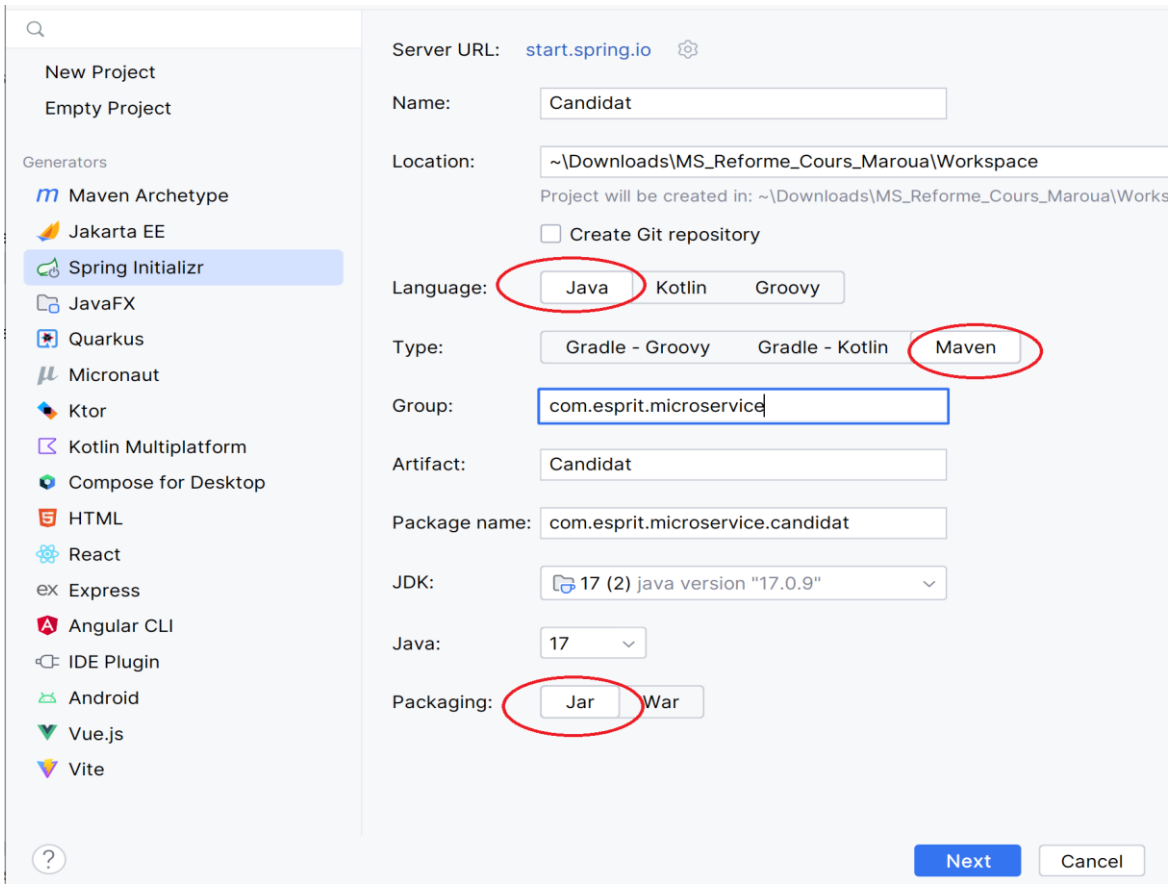
Soit l'entité suivante représentant un candidat:



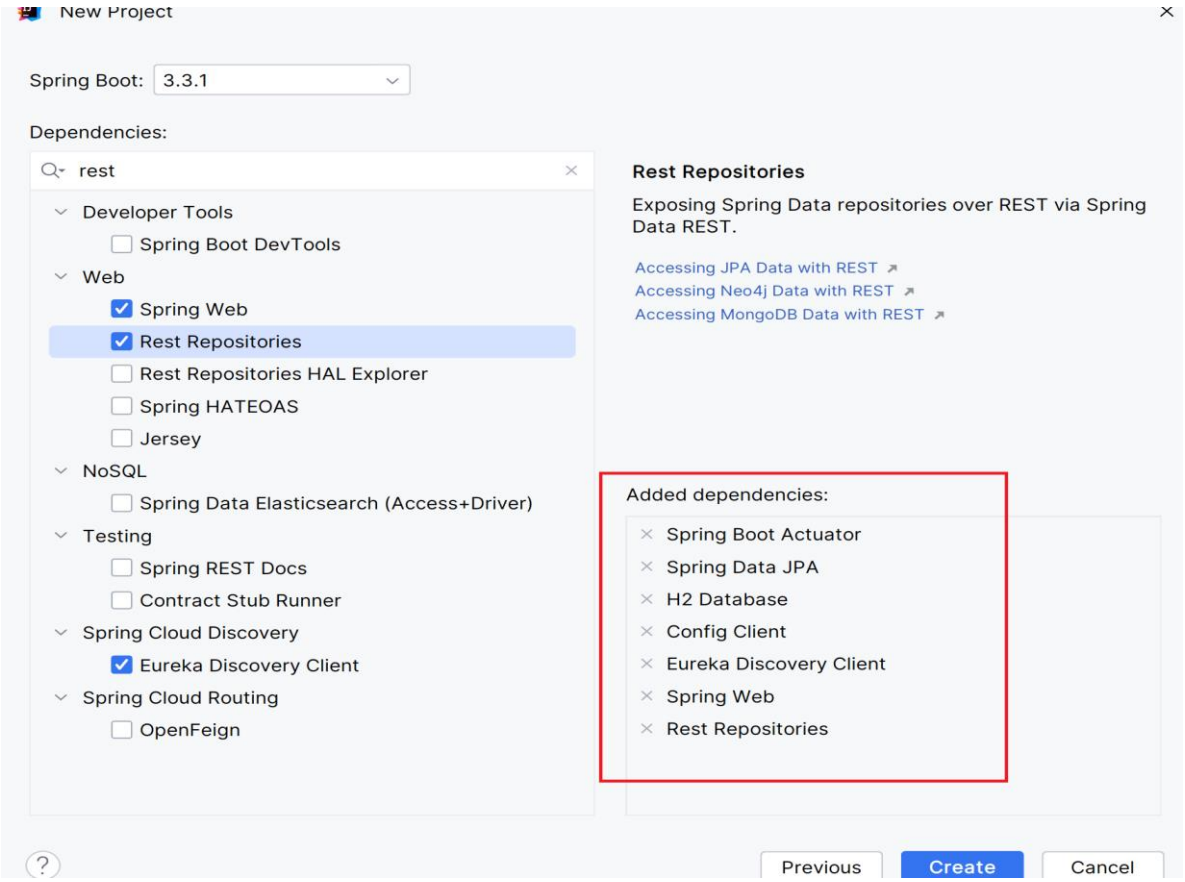
Le microservice Gestion candidats va se charger des fonctionnalités suivantes :

- Afficher tous les candidats
- Afficher un candidat par son id ou son nom
- Ajouter/modifier/supprimer un candidat

1. Créez un projet **de type Spring boot** sur l'IDE **IntelliJ :**



2. Ajouter les dépendances nécessaires :



3. Créez l'entité "**Candidat**" :

```java
@Entity
public class Candidat  {
    1 usage
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private  int id;
    3 usages
    private String nom, prenom, email;

    public int getId() { return id; }
    1 usage
    public String getNom() { return nom; }
    1 usage
    public void setNom(String nom) { this.nom = nom; }
    1 usage
    public String getPrenom() { return prenom; }
    1 usage
    public void setPrenom(String prenom) { this.prenom = prenom; }
    1 usage
    public String getEmail() { return email; }
    1 usage
    public void setEmail(String email) { this.email = email; }

    public Candidat() {
    }

    4 usages
    public Candidat(String nom, String prenom, String email) {
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
    }
}
```

4. Créez le contrôleur REST de notre microservice qui est représenté par une classe nommée "**CandidatRestAPI**".

```java
@RestController
@RequestMapping("/candidats")

public class CandidatRestAPI {
    private String title="Hello, i'm the candidate Micro-Service";

    @RequestMapping("/hello")
    public String sayHello(){
        System.out.println(title);
        return title;
    }
}
```

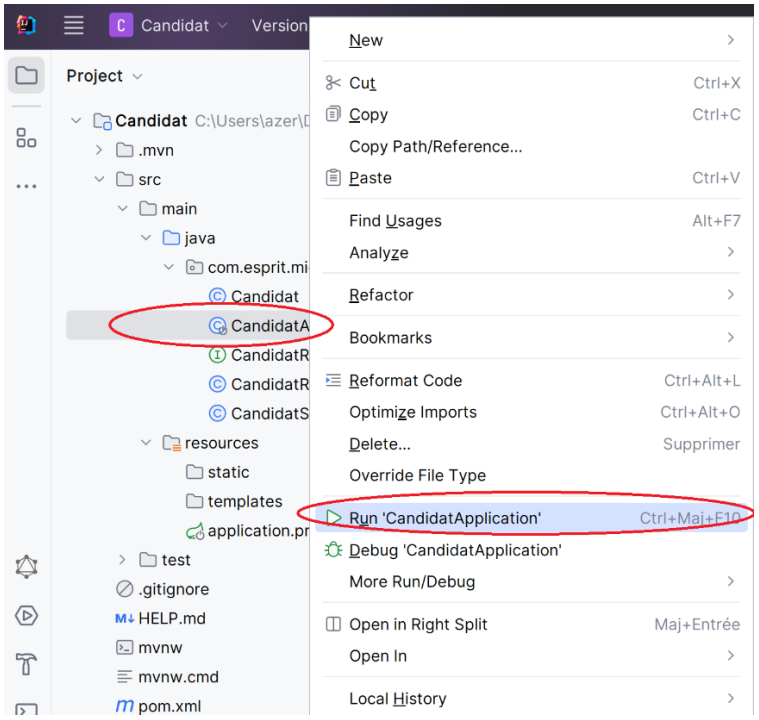
5. Créez le Repository lié à l'entité Candidat :

```java
public interface CandidatRepository extends JpaRepository<Candidat , Integer> {

}
```

6. Créez une fonction dans la classe **CandidatApplication** (créée par défaut lors de la création du projet) qui va permettre d'insérer automatiquement des candidats dans la base

```
@SpringBootApplication
public class CandidatApplication {
   public static void main(String[] args) {
      SpringApplication.run(CandidatApplication.class, args);
   }
   @Autowired
   private CandidatRepository repository;

   @Bean
   ApplicationRunner init() { return
      (args) -> {
         // Vérifier si le repository est vide
         if (repository.count() == 0) {
           repository.save(new Candidat("Mariem", "Ch", "ma@esprit.tn"));
           repository.save(new Candidat("Sarra", "ab", "sa@esprit.tn"));
           repository.save(new Candidat("Mohamed", "ba", "mo@esprit.tn"));
           repository.save(new Candidat("Maroua", "dh", "maroua@esprit.tn"));
         }
         // Affichage
         repository.findAll().forEach(System.out::println);

      };
   }
}
```

7. Exécuter la classe CandidatApplication :



8. Accéder à l'adresse http://localhost:8080/


```
Enregistrer  Copier  Tout réduire  Tout développer  Filtrer le JSON
▼ _links:
  ▼ candidats:
      href:        "http://localhost:8080/candidats{?page,size,sort}"
      templated:   true
  ▼ profile:
      href:        "http://localhost:8080/profile"
```


- ✓ Vous pouvez accéder à l'adresse http://localhost:8080/candidats pour voir les détails de tous les candidats.
- ✓ Vous pouvez également faire une recherche de candidats par id : http://localhost:8080/candidats/4

9. Pour ajouter maintenant une requête de recherche d'un candidat par son nom, on modifie ainsi la classe CandidatRepository:

```
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface CandidatRepository extends JpaRepository<Candidat, Integer> {

    @Query("select c from Candidat c where c.nom like :name")
    public Page<Candidat> candidatByNom(@Param("name") String n, Pageable pageable);
}
```

Lancez l'URL http://localhost:8080/candidats/search/candidatByNom?name=Maroua pour faire la recherche d'un candidat par nom


```
localhost:8080/candidats/search/candidatByNom?name=Maroua
Google  Google  Google  Gmail  YouTube  Maps  Android Telephony...  controler
Impression élégante
{
  "_embedded" : {
    "candidats" : [ {
      "nom" : "Maroua",
      "prenom" : "dh",
      "email" : "maroua@esprit.tn",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8080/candidats/4"
        },
        "candidat" : {
          "href" : "http://localhost:8080/candidats/4"
        }
      }
    } ]
  },
  "_links" : {
    "self" : {
      "href" : "http://localhost:8080/candidats/search/candidatByNom?name=Maroua&page=0&size=20"
    }
  },
  "page" : {
    "size" : 20,
    "totalElements" : 1,
    "totalPages" : 1,
    "number" : 0
  }
}
```

NB : Les données ont été enregistré dans le repository pour un premier déploiement du projet (via repository.save) et affichées ainsi via l'appel aux ressources REST à partir de leurs Endpoints par défaut.

Quel est le SGBD utilisé alors qu'aucune configuration relative à la base de données et au SGBD n'a encore été définie ?

10. Pour l'affichage des détails concernant la BD utilisée par défaut, ajouter le code suivant au sein de votre contrôleur (la classe **CandidatRestAPI**) :

```
@Autowired
DataSource dataSource;

@GetMapping("/db-info")
public String getDbInfo() throws Exception {
    return dataSource.getConnection().getMetaData().getDatabaseProductName();
}
```

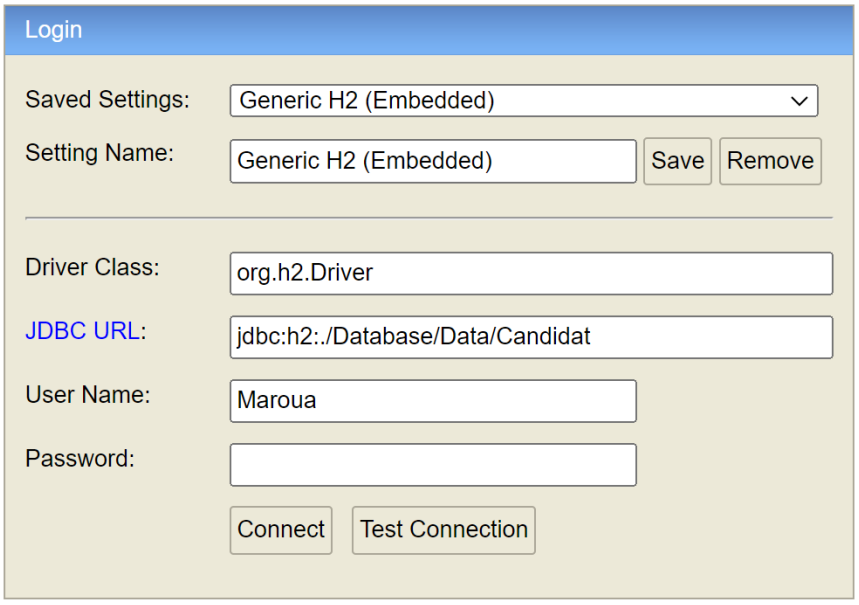
Quand vous ouvrez : http://localhost:8080/candidats/db-info vous obtiendrez :

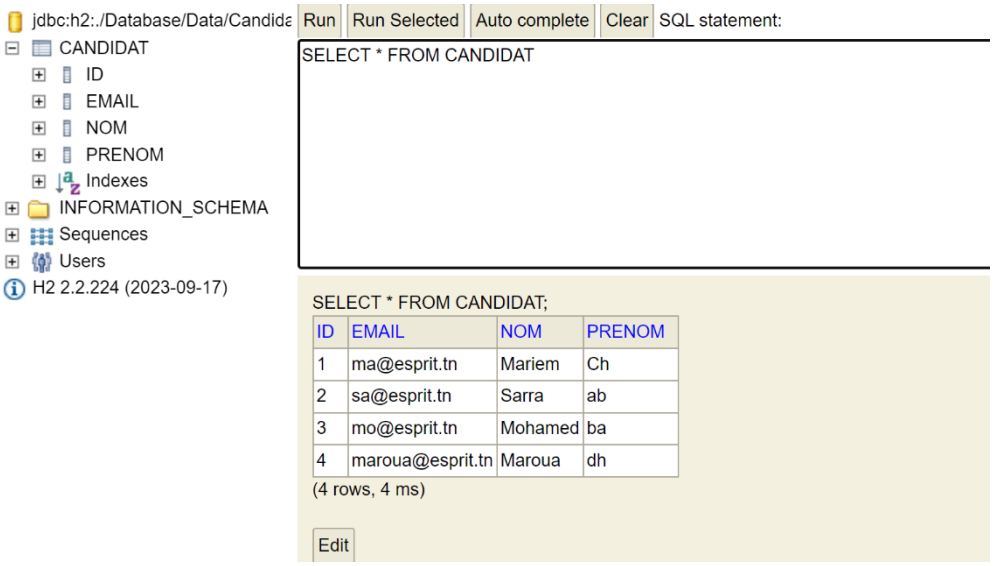
On peut conclure alors que **Spring Boot utilise H2 par défaut** quand aucune base externe n'est configurée.

11. Pour le monitoring de la base de données H2, ajoutez la configuration suivante dans le fichier **application.properties**:

```
# H2
spring.h2.console.enabled=true
spring.h2.console.path=/h2

# Datasource
spring.datasource.username=Maroua
spring.datasource.password=
spring.datasource.url=jdbc:h2:file:./Database/Data/Candidat
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto = update
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

Accédez ensuite à l'URL http://localhost:8080/h2 pour consulter l'interface de gestion de la base H2. Ajouter l'URL d'accès au JDBC **./Database/Data/Candidat** pour voir et traiter les données de votre BD h2. Donner votre userName et password :





12. Nous allons maintenant préparer les méthodes du service Candidat. Ajoutez la classe **CandidatService** qui va se charger d'implémenter ces méthodes:

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class CandidatService {

    @Autowired
    private CandidatRepository candidateRepository;

    public List<Candidat> getAll(){
            return candidateRepository.findAll();
    }

    public Candidat addCandidat(Candidat candidate) {
        return candidateRepository.save(candidate);
    }
```

```java
    public Candidat updateCandidat(int id, Candidat newCandidat) {
        if (candidateRepository.findById(id).isPresent()) {

            Candidat existingCandidat = candidateRepository.findById(id).get();
            existingCandidat.setNom(newCandidat.getNom());
            existingCandidat.setPrenom(newCandidat.getPrenom());
            existingCandidat.setEmail(newCandidat.getEmail());

            return candidateRepository.save(existingCandidat);
        } else
            return null;
    }

    public String deleteCandidat(int id) {
        if (candidateRepository.findById(id).isPresent()) {
            candidateRepository.deleteById(id);
            return "candidat supprimé";
        } else
            return "candidat non supprimé";
    }
}
```

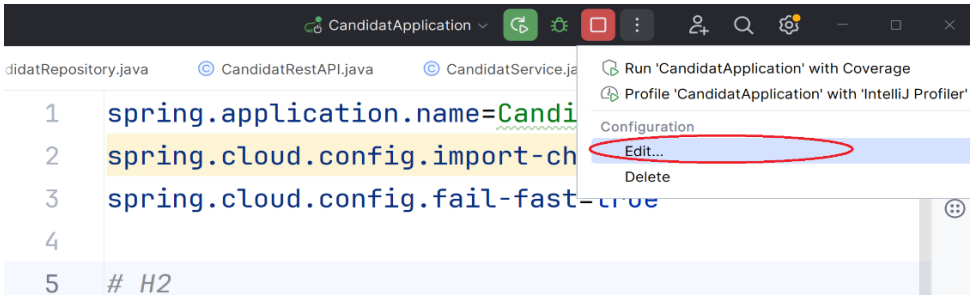
13. Nous préparons par la suite notre API REST. Configurer les requêtes REST dans le contrôleur REST en ajoutant les méthodes qui suivent au sein de la classe **CandidatRestAPI** déjà créée:

```java
 Autowired
 private CandidatService candidatService;


 @GetMapping
 public ResponseEntity<List<Candidat>> getListCandid() {
   List<Candidat> candidats = candidatService.getAll();
   if (candidats.isEmpty()) {
       return ResponseEntity.noContent().build();
   }
     return ResponseEntity.ok(candidats);
}


 @PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)
 public ResponseEntity<Candidat> createCandidat(@RequestBody Candidat candidat) {
   return new ResponseEntity<>(candidatService.addCandidat(candidat),HttpStatus.CREATED);
 }
```

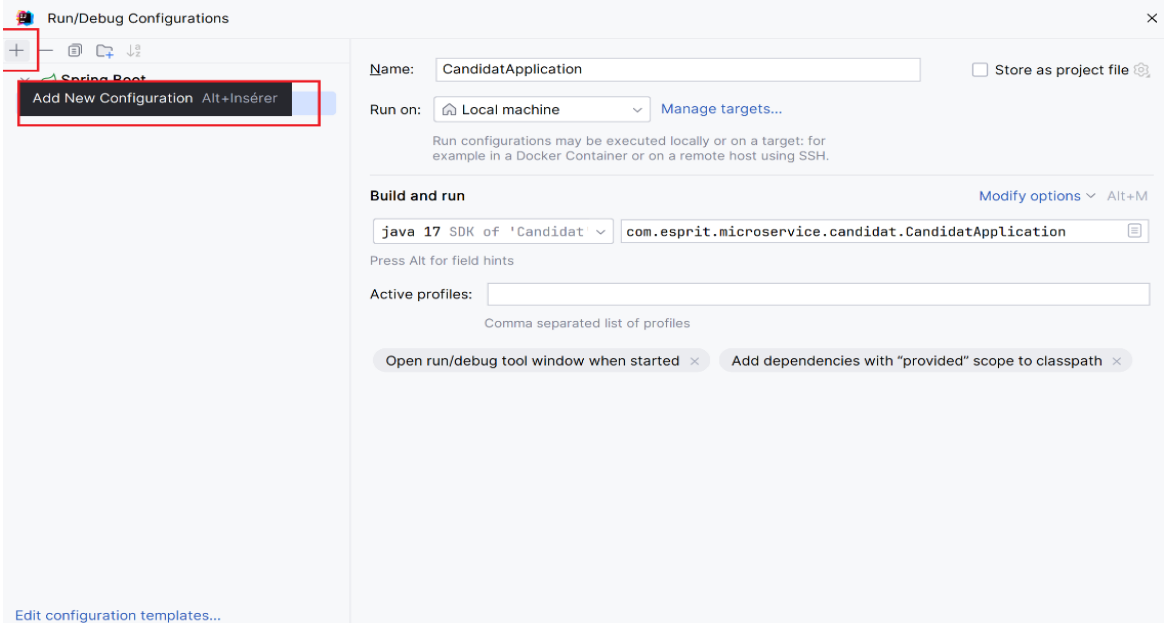
```
 @PutMapping("/{id}")
 public ResponseEntity<Candidat> updateCandidat(@PathVariable("id") int id,
                                                @RequestBody Candidat candidat){
   return ResponseEntity.ok(candidatService.updateCandidat(id, candidat);
 }

@DeleteMapping("/{id}")
public ResponseEntity<String> deleteCandidat(@PathVariable("id") int id){
      return new ResponseEntity<>(candidatService.deleteCandidat(id), HttpStatus.OK);
}
```

14. Afin de lancer une **deuxième instance** du même service sur un port différent :

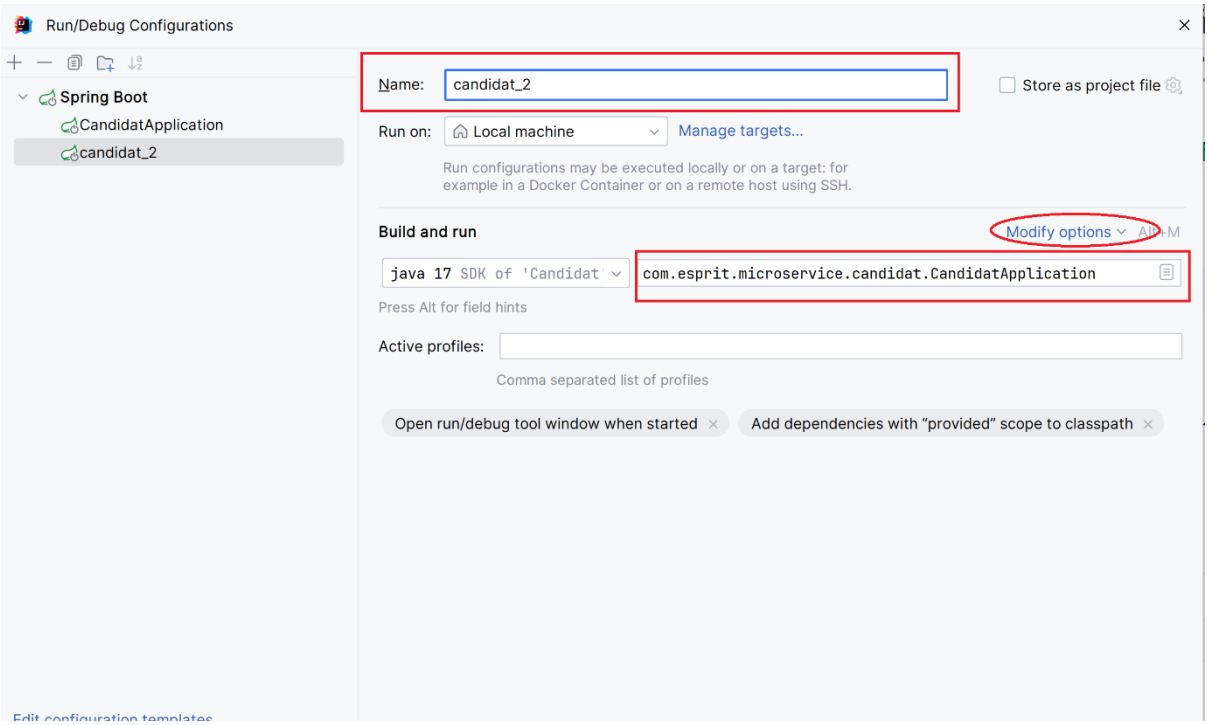
a. accédez à **Edit Configuration** :



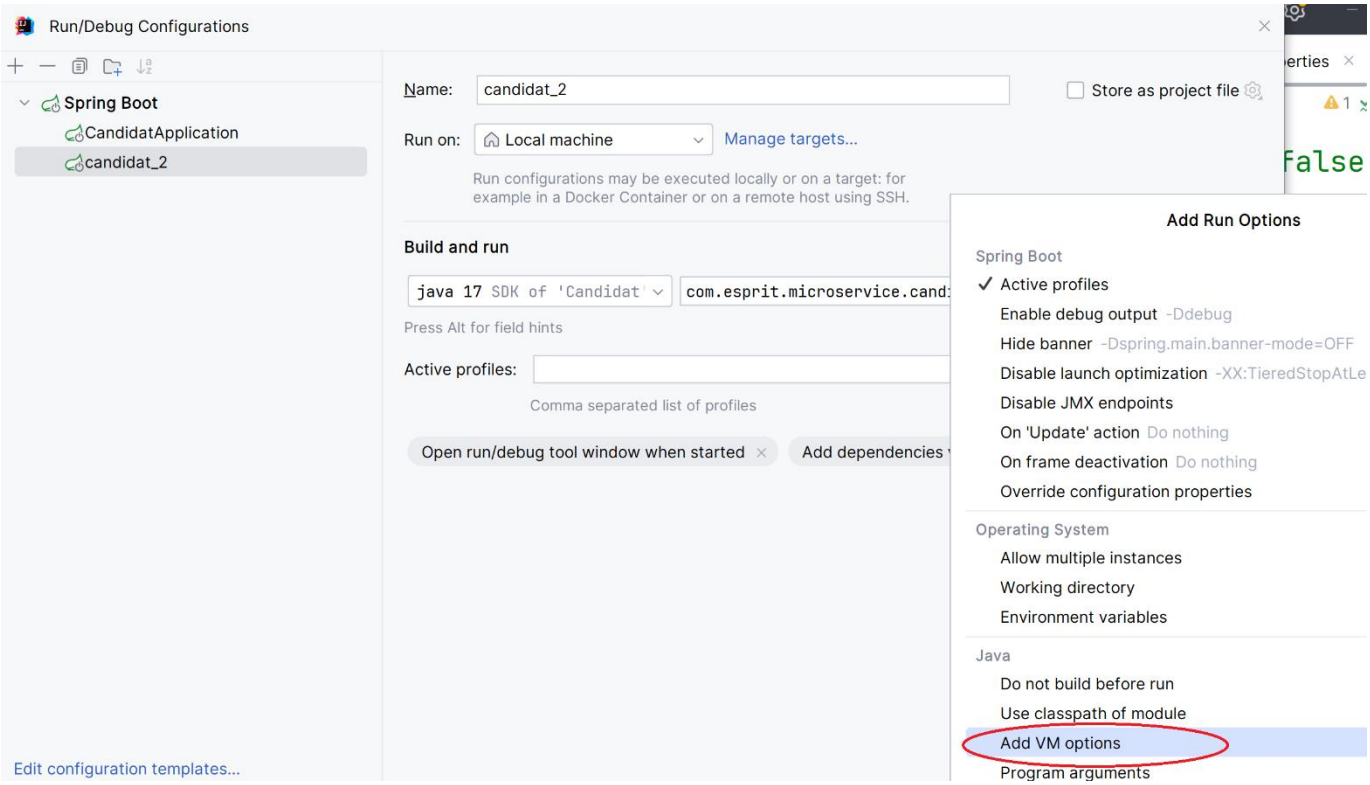
b. Cliquez sur « **Add New Configuration** » :



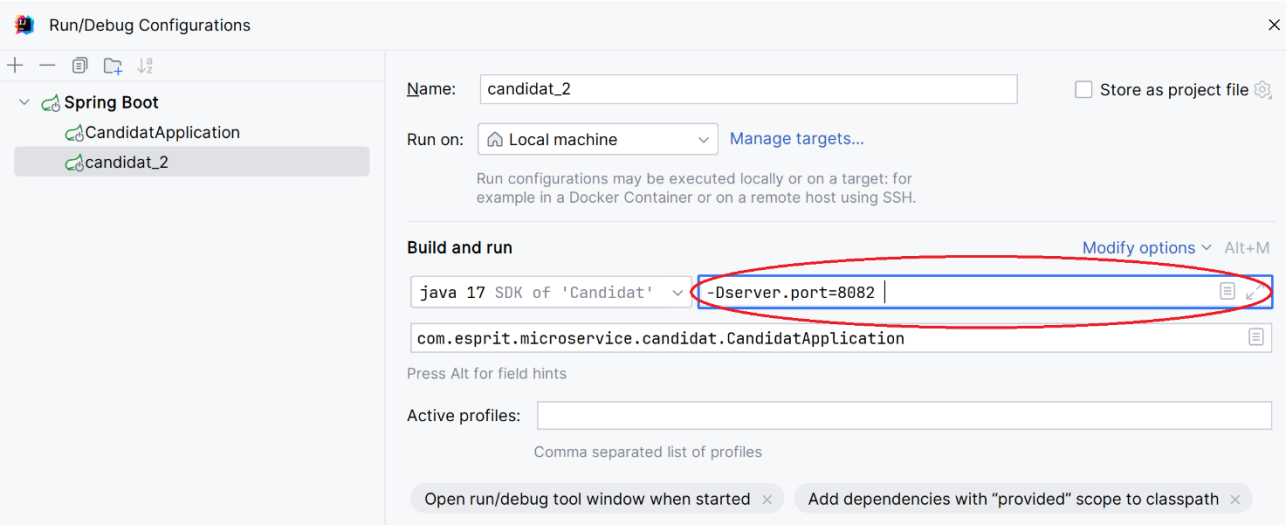
c. Choisissez « **Spring Boot** » puis ajouter la configuration suivante en choisissant la classe CandidateApplication comme « **Main class** ».



d. Cliquez sur « **Modify options** » --> « **Add VM options** » :



e. Fixez le port de la nouvelle instance, du même service, en ajoutant l'arguement : `-Dserver.port = 8082` :



15. Exécutez la nouvelle instance et y accéder à travers l'URL « http://localhost:8082/

** Ne pas arrêter l'ancienne instance du service s'exécutant sur le port 8080.

L'erreur suivante sera affichée :

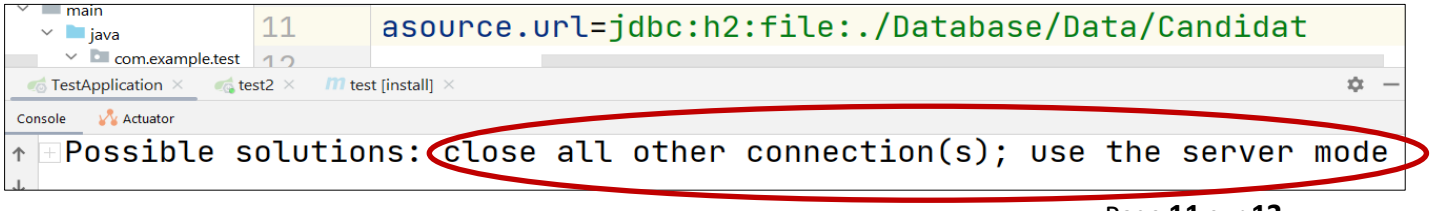
```
m.core                      : HHH000489: No JTA platform available (set 'hibernate.tra
EntityManagerFactoryBean    : Initialized JPA EntityManagerFactory for persistence uni
QueryEnhancerFactories      : Hibernate is in classpath; If applicable, HQL parser wil
tion$JpaWebConfiguration    : spring.jpa.open-in-view is enabled by default. Therefore
i.HikariDataSource          : HikariPool-1 - Starting...
TomcatWebServer             : Tomcat started on port 8080 (http) with context path '/'
.TestApplication            : Started TestApplication in 5.483 seconds (process runnin
i.HikariDataSource          : HikariPool-1 - Starting...
m.jdbc.error                : HHH000247: ErrorCode: 90020, SQLState: 90020
m.jdbc.error                : La base de données est peut-être en cours d'utilisation:
```

Problème détecté : La 2e instance Spring Boot essaie d'ouvrir directement le fichier Candidat.mv.db alors que la 1re instance l'a déjà verrouillé.

→ **H2 bloque la 2e pour éviter la corruption**.

Solution :

i. Le message clé est déjà affiché dans la console « **Use the Server mode** »:



ii. Dans le fichier application.properties, ajouter **AUTO_SERVER=TRUE** à la fin du `datasource.url` :

```
spring.datasource.url=jdbc:h2:file:./Database/Data/Candidat;AUTO_SERVER=TRUE
```

AUTO_SERVER=TRUE permet à plusieurs applications locales d'utiliser le même fichier H2 en toute sécurité sans corruption ni serveur externe.

iii. Un autre problème à rencontrer quand : `AUTO_SERVER=TRUE` et `spring.jpa.hibernate.ddl-auto = update`

Si ddl-auto=update est activé → les deux instances essaient de **modifier le schéma en même temps** → risque de deadlocks ou corruption ;

iv. Pour que toutes les instances se connectent **en sécurité** à la même base en partageant le même fichier .mv.db MAIS **sans jamais toucher le schéma de la BD : il faut que** `spring.jpa.hibernate.ddl-auto = none`

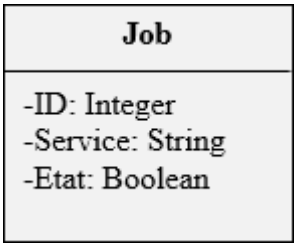
v. Avec `ddl-auto = none`, aucune modification du schéma par Hibernate ne sera autorisée.

La configuration finale de SGBD H2 est alors la suivante :

```
# Datasource
spring.datasource.username=Maroua
spring.datasource.password=
spring.datasource.url=jdbc:h2:file:./Database/Data/Candidat;AUTO_SERVER=TRUE
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto = none
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

Travail à faire :

Implémentez un deuxième microservice (dans un nouveau projet spring boot) qui se charge de la gestion des jobs et utilisant **MySQL** comme Base de données. L'entité Job est représentée comme suit:



Ce deuxième microservice prendra en charge les fonctionnalités suivantes:

- Afficher tous les jobs
- Afficher un job par son id ou son nom
- La modification de l'état de poste :
    - Etat= oui (si poste disponible)
    - Etat= non (si poste est occupé).